

1. INTRODUCTION

This paper presents an efficient FPGA based low power pipelined 64-bit RISC processor with Floating Point Unit. RISC is a design philosophy where it reduces the complexity of the instruction set, which will reduce the amount of space, time, cost, power and heat etc.,. This processor is developed especially for Arithmetic operations of both fixed and floating point numbers, branch and logical functions. Pipelining would not flush when branch instruction occurs as it is implemented using dynamic branch prediction. This will increase flow in instruction pipeline and high effective performance. In RTL coding one can reduce the dynamic power by using clock gating technique. In this paper also implement Double Precision floating point arithmetic operations like addition, subtraction, multiplication and division. This architecture has become indispensable and increasingly important in many applications like signal processing, graphics and medical by using floating point operations. The necessary code is written in the hardware description language Verilog HDL. Quartus II 10.1 suite is used for software development, Modelsim is used for simulations and the design is implemented on Altera's Cyclone DEII FPGA..

In conventional approach the system consumes too much of power. The power reductions in conventional RISC processors are done At fabrication step itself, but which is too complex process. Here the utilization of chip area is more and the system consumes more power which leads to increased latency. To overcome this disadvantage, low power RISC architecture is designed with less number of gates. Low power design means reducing the power consumption.

Low power consumption helps to reduce the heat dissipation, lengthen battery life and increase device reliability. This technology strongly affects battery size, design, electronic packaging of ICs, heat dissipation and circuit reliability. Low power embedded processors are used in a wide variety of applications including cars, mobile phones, digital cameras, printers and other devices. Low power has emerged as a principle theme in today's electronics industry. The need for low power has caused a major paradigm shift where power dissipation has become an important consideration as performance and area. RISC is termed as Reduced Instruction Set Computer [1].

Now a days RISCs are wide spread in all type of computational tasks. In the area of scientific computing RISC workstation is being increasingly used to compute intensive task such as digital signal and image processing [2].

Pipelined RISC is an evolution in computer architecture. It emphasizes on speed and cost effectiveness over the ease of hardware description language programming and conservation of memory. RISC based designs will continue to grow more rapidly than CSIC (Complex Instruction Set

Computer) based designs in case of speed and ability [3]. A standard feature in RISC processors is pipelining, because of this the processor works on different steps of the instruction at the same time, so that more instructions can be executed in a shorter period of time. They are also less costly to design, and manufacture.

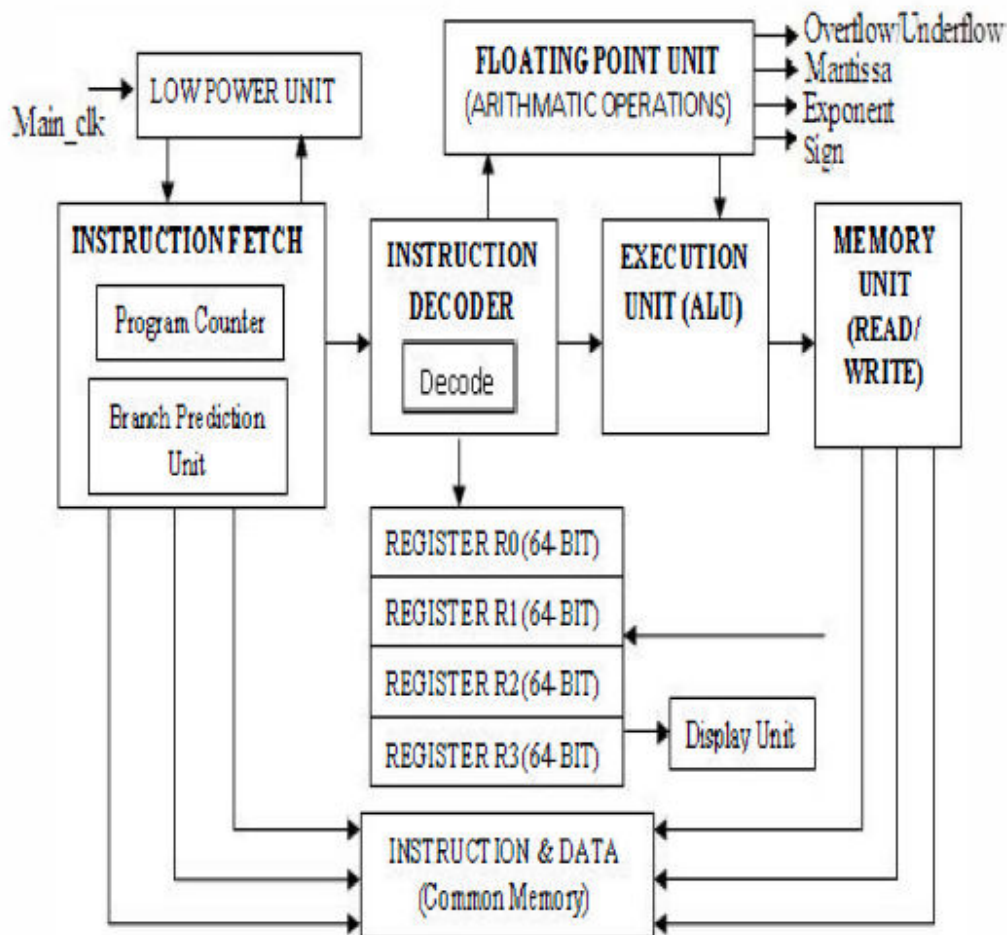
This paper describes low power design of 64-bit data width RISC processor and also a high speed floating point double precision addition, subtraction, multiplication and division operations, which are implemented using pipelined architecture. Through this, one can improve the speed of the operation as well as overall performance. In this design, the pipelining technique consists of four stages. They are Fetch, Decode, Execute and Memory Read/Write [4].

In this paper, the architecture doesn't need any control hazards, as auto branch prediction is happening in the Fetch stage. Without branch prediction, the processor has to wait until the conditional jump has passed the execute cycle before the next instruction can enter the fetch stage in instruction pipeline. The branch predictor attempts to avoid the waste of time whether the conditional jwnp is most likely to be taken or not taken. The branch prediction part to be the most likely is then fetched and speculatively executed. This will increase flow in instruction pipeline and achieve high effective performance. During the design process various low power techniques in architectural level are included. It has a complete instruction set, program and data memories, general purpose registers and a simple Arithmetical Logical Unit (ALU) including Floating Point operations. In this design, most instructions are of uniform length and similar structure.

2. ARCHITECTURE OF THE DESIGN

The architecture of the proposed low power pipelined 64-bit RISC processor [5] with FPU is a single cycle pipelined processor. It has small instruction set, load/store architecture, fixed length coding and hardware decoding and large register set. This is a general-purpose 64-bit RISC processor with pipelining architecture. It gets instructions on a regular basis using dedicated buses to its memory, executes all its native instruction in stages with pipelining. In the low power RISC design, all the arithmetic, branch, logical and floating point arithmetic (add, sub, mul and div) operations are performed and the resultant value is stored in the memory/register and

retrieved back from memory, when required. In the design, power reduction is done in front end process so that low power RISC processor is designed without any complexity. The system architecture of a low power pipelined 64-bit RISC processor with FPU is shown in Fig. 1. The architecture comprises of Modified Harvard Architecture, low power unit and floating unit. The Modified Harvard architecture consists of four stage pipelining: Instruction Fetch, Instruction Decode, Execution Unit and Memory Read/Write. Pipelining technique allows for simultaneous execution of parts or stages of instructions more efficiently [6]. With a RISC processor, one instruction is executed while the next is being decoded and its operands are being loaded while the following instruction is being fetched at the same time. Pipelining would not flush when branch instruction occurs as it is implemented using dynamic branch prediction. The branch prediction attempts to avoid the waste of time whether the conditional jwnp is most likely to be taken or not taken.



2.1 DESCRIPTION OF LOGIC BLOCKS

In the present work, the RISC processor consists of blocks namely, Instruction Fetch (Program Counter), Control Unit, Register File, Arithmetic & Logical Unit(ALU), Floating Point Unit and Memory Unit.

2.1.1 INSTRUCTION FETCH

This stage consists of Program Counter (PC) and Branch prediction. Program Counter which performs two operations, namely, incrementing and loading. The PC contains the address of the instruction that will be fetched from the instruction memory during the next cycle. Normally, the PC is incremented by one instruction during each clock cycle unless a branch instruction is executed. When a branch instruction is encountered, the PC is incremented by the amount indicated by the branch offset. The PC Write input serves as an enable signal. When PC Write signal is high, the contents of the PC are incremented during the next clock cycle. When it is low, the contents of the PC remain unchanged.

The present architecture uses dynamic branch prediction as it reduces branch penalties under hardware control [7]. The prediction is made in Instruction Fetch stage of the pipeline. Thus branch prediction buffer is indexed by the lower order bits of the branch address in Instruction Fetch. It is low for branch not taken and high for branch taken. The branch target can be accessed as soon as the branch target address is computed. Branch Target Cache (BTC) is a branch prediction buffer with additional information as it has an address tag of a branch instruction and stores the target address. Thus BTC determines the target address, if the branch instruction is taken. If these requirements are met, the processor can initiate the next instruction access as soon as the previous access is complete. Thus the main operation of BTC is that during the IF stage, the LSBs of the PC are used to access the BTC and if the MSBs of the PC match the target then the

entry is valid. If the branch is predicted as taken, the predicted target address is used to access during the next cycle.

2.2.2 CONTROL UNIT

The control unit generates all the control signals needed to control the coordination among the entire component of the processor. This unit generates signals that control all the read and write operation of the register file and the data memory. It is also responsible for generating signals that decide when to use the multiplier and when to use the ALU. It generates appropriate branch flags that are used by the Branch Decide unit.

2.2.3 REGISTER FILE

This is a two port register file which can perform two simultaneous read and write operations. It contains four 64-bit general purpose registers. These register files are utilized during the arithmetic, data instructions and floating point operations. It can be addressed as both source and destination using a 2-bit identifier. The registers are named as RO through R3. The load instruction is used to load the values into the registers and store instruction is used to hold the address of the corresponding memory locations. When the Reg_Write signal is high a write operation is performed to the register.

2.2.4 ARITHMETIC LOGIC UNIT

The ALU is responsible for arithmetic and logic operations that take place within the processor. These operations can have one operand or two, these values coming from either the register file or from the immediate value from the instruction directly. The operations supported by the ALU include add, sub, compare, increment, AND, OR, NOT, NAND and NOR. The output of the ALU goes either to the data memory or through a multiplexer back to the register file. The multiplier is designed to execute in a single cycle instructions. All

operations will be done according to the control signal coming from ALU control unit.

Control unit is responsible for providing signals to the ALU that indicates the operation that the ALU will perform. The input to this unit is the 5-bit opcode and the 2-bit function field of the instruction word. It uses these bits to decide the correct that is used to gate the signals to the parts of the ALU that it will not be using for the current operation. This stage consists of some control circuitry that forwards the appropriate data, generated by the ALU or read from the data memory to the register files to be written into the designated register.

2.2.5 FLOATING POINT UNIT

A floating point (FPU), also known as a math co-processor or numeric processor is a specialized co-processor that manipulates numbers more quickly than the basic microprocessor circuitry. The FPU does this by means of instructions that focus entirely on large mathematical operations. Floating point computational logic has long been a mandatory component of high performance computer systems as well as embedded systems and mobile applications. The performance of many modern applications which give a high frequency of floating point operations is often limited by the speed of the floating point hardware. The advantage of floating point representation over fixedpoint and integer representation is that it can support a much wider range of values. In the present work 64-bit FPU is incorporated, which supports double precision IEEE-754 format. The IEEE-754 standard defines a double as 1 bit for sign, 11 bits for exponent and 53 bits (52 explicitly stored) for mantissa [8]. This FPGA implementation of 64-bit double precision floating point has been proposed in this paper which performs certain operations like addition, subtraction, multiplication and division. This kind of unit can be tremendously useful in the FPGA implementation of complex systems that benefits from the parallelism of the FPGA device [9].

FP Add: In the module FP Add, the inputs operands are separated into their mantissa and exponent components. Then the exponents are compared to check which variable is larger. The larger variable goes into "mantissaLarge" and exponent_large". Similarly the smaller variable goes into "mantissa_small" and "exponent_small". The sign and exponent of the output will be determined; the smaller exponent can be right shifted before performing the addition.

FP Sub: The input variables are separated into two components namely mantissa and exponent. Subtraction is similar to that of addition such that the mantissa of the smaller exponent is shifted to the right before performing the subtraction [10].

FP Mul: Multiplying all 53 bits of var1 by 53 bits of var2 would result in a 106-bit product. 53 bit by 53 bit multipliers are not available in the Altera FPGAs, so the multiply would be broken down into smaller multiplies and the results would be added together to give the final 106-bit product. The module (FP Mul) breaks up the multiply which can perform 24-bit by 17-bit.

FP Div: Division is performed in FP Div. The exponent is obtained by adding 1023 with the exponent of var1 and then by subtracting the exponent of var2 from this sum. Then, the mantissa of var1 is the dividend and the mantissa of var2 is the divisor.

2.2.6 MEMORY UNIT

The load and store instructions are used to access this module. Finally, the memory access stage is where, if necessary, system memory is accessed for data. Also if a write to the data memory is required by the instruction it is done in this stage. In order to avoid additional complications it is assumed that a single read or write is accomplished within a single CPU clock cycle.

2.2.7 INSTRUCTIONS SET

The instruction set used in this architecture consists of arithmetic, logical, memory and branch instructions. It will have short (8-bit) and long (16-bit) instructions, which are

shown in Table 1. For all arithmetic & logical operations, 8-bit instructions are used. For all memory transactions and jump instructions, 16-bit instructions are used. It will have special instructions to access external ports. The architecture will also have 64-bit general purpose registers that can be used in all operations. For all the jump instruction, the processor architecture will automatically flush the data in the pipeline, so as to avoid any misbehavior.

TABLE I. INSTRUCTION SET

Short Instruction Format:

Opcode Source

1010 10

Long Instruction Format:

Opcode Source

0011 00

Address

0101 11

H Low Power Technique

Destination

11

Destination

??

01

There are several different RTL and gate-level design strategies for reducing power. In the present work, Clock Gating design is used for reducing dynamic power. In this method, clock is applied to only the modules that are working at that instant [11]. Clock gating is a dynamic power reduction method in which the clock signals are stopped for selected registers banks during the time when the stored logic values are not changing.

2.2.7 LOW POWER TECHNIQUE

The clock pulse for low power technique is shown in Fig. 2.

The input to low power unit is global clock and its output is gated clock, since the module will block the main clock in the following conditions.

1. When instruction is halt.
2. When there is a continuous Nop operation.
3. When program counter fails to increment.

4. XILINX SOFTWARE

4.1 INTRODUCTION

Xilinx ISE^[2] (Integrated Synthesis Environment)^[3] is a software tool produced by [Xilinx](#) for synthesis and analysis of [HDL](#) designs, enabling the developer to [synthesize](#) ("compile") their designs, perform [timing analysis](#), examine [RTL](#) diagrams, simulate a design's reaction to different stimuli, and configure the target device with the [programmer](#).

Xilinx ISE is a design environment for FPGA products from Xilinx, and is tightly-coupled to the architecture of such chips, and cannot be used with FPGA products from other vendors.^[3] The Xilinx ISE is primarily used for circuit synthesis and design, while ISIM or the [ModelSim](#) logic simulator is used for system-level testing.^{[4][5]} Other components shipped with the Xilinx ISE include the Embedded Development Kit (EDK), a Software Development Kit (SDK) and ChipScope Pro.^[6]

Since 2012, Xilinx ISE has been discontinued in favor of [Vivado Design Suite](#), that serves the same roles as ISE with additional features for [system on a chip](#) development.^{[7][8]} Xilinx released the last version of ISE in October 2013 (version 14.7), and states that "ISE has moved into the sustaining phase of its product life cycle, and there are no more planned ISE releases."

4.2 USER INTERFACE

The primary user interface of the ISE is the Project Navigator, which includes the design hierarchy (Sources), a [source code](#) editor (Workplace), an output console (Transcript), and a processes tree (Processes).^{[3][9]}

The Design hierarchy consists of design files (modules), whose dependencies are interpreted by the ISE and displayed as a [tree structure](#).^[3] For single-chip designs there may be one main module, with other modules included by the main module, similar to the `main()` subroutine in [C++](#) programs.^[3] Design constraints are specified in modules, which include pin configuration and mapping.^[3]

The Processes hierarchy describes the operations that the ISE will perform on the currently active module.^[3] The hierarchy includes compilation functions, their dependency functions, and other utilities.^[3] The window also denotes issues or errors that arise with each function.^[3]

The Transcript window provides status of currently running operations, and informs engineers on design issues.^[3] Such issues may be filtered to show Warnings, Errors, or both.

4.3 SIMULATION

System-level testing may be performed with ISIM or the [ModelSim](#) logic simulator, and such test programs must also be written in HDL languages.^[3] Test bench programs may include simulated input signal waveforms, or monitors which observe and verify the outputs of the [device under test](#).^[3]

[ModelSim](#) or ISIM may be used to perform the following types of simulations:^[4]

- Logical verification, to ensure the module produces expected results
- Behavioural verification, to verify logical and timing issues
- Post-place & route simulation, to verify behaviour after placement of the module within the reconfigurable logic of the FPGA.
 - Xilinx's patented algorithms for synthesis allow designs to run up to 30% faster than competing programs, and allows greater logic density which reduces project time and costs.^[10]
 - Also, due to the increasing complexity of FPGA fabric, including memory blocks and I/O blocks, more complex synthesis algorithms were developed that separate unrelated modules into *slices*, reducing post-placement errors.^[10]

- IP Cores are offered by Xilinx and other third-party vendors, to implement system-level functions such as [digital signal processing](#) (DSP), bus interfaces, networking protocols, [image processing](#), [embedded processors](#), and peripherals.^[10] Xilinx has been instrumental in shifting designs from ASIC-based implementation to FPGA-based implementation.

5. CODING

VHDL code for MIPS Processor

VHDL code for Data Memory of the MIPS processor:

-- VHDL project: VHDL code for single-cycle MIPS Processor

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

USE IEEE.numeric_std.all;

-- VHDL code for the data Memory of the MIPS Processor

entity Data_Memory_VHDL is

port (

clk: in std_logic;

mem_access_addr: in std_logic_Vector(15 downto 0);

mem_write_data: in std_logic_Vector(15 downto 0);

mem_write_en,mem_read:in std_logic;

mem_read_data: out std_logic_Vector(15 downto 0)

);

end Data_Memory_VHDL;

architecture Behavioral of Data_Memory_VHDL is

signal i: integer;

```

signal ram_addr: std_logic_vector(7 downto 0);
type data_mem is array (0 to 255 ) of std_logic_vector (15 downto 0);
signal RAM: data_mem :=((others=> (others=>'0')));
begin

    ram_addr <= mem_access_addr(8 downto 1);

    process(clk)
    begin
        if(rising_edge(clk)) then
            if (mem_write_en='1') then
                ram(to_integer(unsigned(ram_addr))) <= mem_write_data;
            end if;
        end if;
    end process;

    mem_read_data <= ram(to_integer(unsigned(ram_addr))) when (mem_read='1') else x"0000";

end Behavioral;

```

VHDL code for ALU of the MIPS processor:

```

-- VHDL project: VHDL code for single-cycle MIPS Processor

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.STD_LOGIC_signed.all;

-- VHDL code for ALU of the MIPS Processor

entity ALU_VHDL is

```

```

port(
    a,b : in std_logic_vector(15 downto 0); -- src1, src2
    alu_control : in std_logic_vector(2 downto 0); -- function select
    alu_result: out std_logic_vector(15 downto 0); -- ALU Output Result
    zero: out std_logic -- Zero Flag
);
end ALU_VHDL;

```

architecture Behavioral of ALU_VHDL is

```

signal result: std_logic_vector(15 downto 0);

```

```

begin

```

```

    process(alu_control,a,b)

```

```

    begin

```

```

        case alu_control is

```

```

            when "000" =>

```

```

                result <= a + b; -- add

```

```

            when "001" =>

```

```

                result <= a - b; -- sub

```

```

            when "010" =>

```

```

                result <= a and b; -- and

```

```

            when "011" =>

```

```

                result <= a or b; -- or

```

```

            when "100" =>

```

```

                if (a<b) then

```

```

                    result <= x"0001";

```

```

else
    result <= x"0000";
end if;
when others => result <= a + b; -- add
` end case;
end process;

zero <= '1' when result=x"0000" else '0';

alu_result <= result;
end Behavioral;

```

VHDL code for ALU Control Unit of the MIPS processor:

```

-- VHDL project: VHDL code for single-cycle MIPS Processor

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- VHDL code for ALU Control Unit of the MIPS Processor
entity ALU_Control_VHDL is
port(
    ALU_Control: out std_logic_vector(2 downto 0);
    ALUOp : in std_logic_vector(1 downto 0);
    ALU_Funct : in std_logic_vector(2 downto 0)
);
end ALU_Control_VHDL;

architecture Behavioral of ALU_Control_VHDL is
begin

```

```

process(ALUOp,ALU_Funct)
begin
case ALUOp is
when "00" =>
    ALU_Control <= ALU_Funct(2 downto 0);
when "01" =>
    ALU_Control <= "001";
when "10" =>
    ALU_Control <= "100";
when "11" =>
    ALU_Control <= "000";
when others => ALU_Control <= "000";
end case;
end process;
end Behavioral;

```

VHDL code for Register File of the MIPS processor:

```

-- VHDL project: VHDL code for single-cycle MIPS Processor

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
USE IEEE.numeric_std.all;

-- VHDL code for the register file of the MIPS Processor

entity register_file_VHDL is
port (
    clk,rst: in std_logic;

```

```

reg_write_en: in std_logic;
reg_write_dest: in std_logic_vector(2 downto 0);
reg_write_data: in std_logic_vector(15 downto 0);
reg_read_addr_1: in std_logic_vector(2 downto 0);
reg_read_data_1: out std_logic_vector(15 downto 0);
reg_read_addr_2: in std_logic_vector(2 downto 0);
reg_read_data_2: out std_logic_vector(15 downto 0)
);
end register_file_VHDL;

```

architecture Behavioral of register_file_VHDL is

```

type reg_type is array (0 to 7 ) of std_logic_vector (15 downto 0);
signal reg_array: reg_type;
begin
    process(clk,rst)
    begin
        if(rst='1') then
            reg_array(0) <= x"0001";
            reg_array(1) <= x"0002";
            reg_array(2) <= x"0003";
            reg_array(3) <= x"0004";
            reg_array(4) <= x"0005";
            reg_array(5) <= x"0006";
            reg_array(6) <= x"0007";
            reg_array(7) <= x"0008";

```

```
elsif(rising_edge(clk)) then
```

```
  if(reg_write_en='1') then
```

```
    reg_array(to_integer(unsigned(reg_write_dest))) <= reg_write_data;
```

```
  end if;
```

```
end if;
```

```
end process;
```

```
reg_read_data_1 <= x"0000" when reg_read_addr_1 = "000" else  
reg_array(to_integer(unsigned(reg_read_addr_1)));
```

```
reg_read_data_2 <= x"0000" when reg_read_addr_2 = "000" else  
reg_array(to_integer(unsigned(reg_read_addr_2)));
```

```
end Behavioral;
```

VHDL code for Control Unit of the MIPS processor:

```
-- VHDL project: VHDL code for single-cycle MIPS Processor
```

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
-- VHDL code for Control Unit of the MIPS Processor
```

```
entity control_unit_VHDL is
```

```
port (
```

```
  opcode: in std_logic_vector(2 downto 0);
```

```
  reset: in std_logic;
```

```
  reg_dst,mem_to_reg,alu_op: out std_logic_vector(1 downto 0);
```

```
  jump,branch,mem_read,mem_write,alu_src,reg_write,sign_or_zero: out std_logic
```

```
);
```

```
end control_unit_VHDL;
```

architecture Behavioral of control_unit_VHDL is

```
begin
```

```
process(reset,opcode)
```

```
begin
```

```
if(reset = '1') then
```

```
    reg_dst <= "00";
```

```
    mem_to_reg <= "00";
```

```
    alu_op <= "00";
```

```
    jump <= '0';
```

```
    branch <= '0';
```

```
    mem_read <= '0';
```

```
    mem_write <= '0';
```

```
    alu_src <= '0';
```

```
    reg_write <= '0';
```

```
    sign_or_zero <= '1';
```

```
else
```

```
case opcode is
```

```
when "000" => -- add
```

```
    reg_dst <= "01";
```

```
    mem_to_reg <= "00";
```

```
    alu_op <= "00";
```

```
    jump <= '0';
```

```
branch <= '0';

mem_read <= '0';

mem_write <= '0';

alu_src <= '0';

reg_write <= '1';

sign_or_zero <= '1';

when "001" => -- sliu

reg_dst <= "00";

mem_to_reg <= "00";

alu_op <= "10";

jump <= '0';

branch <= '0';

mem_read <= '0';

mem_write <= '0';

alu_src <= '1';

reg_write <= '1';

sign_or_zero <= '0';

when "010" => -- j

reg_dst <= "00";

mem_to_reg <= "00";

alu_op <= "00";

jump <= '1';

branch <= '0';

mem_read <= '0';

mem_write <= '0';
```

```
alu_src <= '0';  
reg_write <= '0';  
sign_or_zero <= '1';  
when "011" =>-- jal  
    reg_dst <= "10";  
    mem_to_reg <= "10";  
    alu_op <= "00";  
    jump <= '1';  
    branch <= '0';  
    mem_read <= '0';  
    mem_write <= '0';  
    alu_src <= '0';  
    reg_write <= '1';  
    sign_or_zero <= '1';  
when "100" =>-- lw  
    reg_dst <= "00";  
    mem_to_reg <= "01";  
    alu_op <= "11";  
    jump <= '0';  
    branch <= '0';  
    mem_read <= '1';  
    mem_write <= '0';  
    alu_src <= '1';  
    reg_write <= '1';  
    sign_or_zero <= '1';
```

```
when "101" => -- sw
    reg_dst <= "00";
    mem_to_reg <= "00";
    alu_op <= "11";
    jump <= '0';
    branch <= '0';
    mem_read <= '0';
    mem_write <= '1';
    alu_src <= '1';
    reg_write <= '0';
    sign_or_zero <= '1';
when "110" => -- beq
    reg_dst <= "00";
    mem_to_reg <= "00";
    alu_op <= "01";
    jump <= '0';
    branch <= '1';
    mem_read <= '0';
    mem_write <= '0';
    alu_src <= '0';
    reg_write <= '0';
    sign_or_zero <= '1';
when "111" => -- addi
    reg_dst <= "00";
    mem_to_reg <= "00";
```

```

alu_op <= "11";
jump <= '0';
branch <= '0';
mem_read <= '0';
mem_write <= '0';
alu_src <= '1';
reg_write <= '1';
sign_or_zero <= '1';
when others =>
    reg_dst <= "01";
    mem_to_reg <= "00";
    alu_op <= "00";
    jump <= '0';
    branch <= '0';
    mem_read <= '0';
    mem_write <= '0';
    alu_src <= '0';
    reg_write <= '1';
    sign_or_zero <= '1';
end case;
end if;
end process;

end Behavioral;

```

VHDL code for Instruction Memory of the MIPS processor:

```

-- VHDL project: VHDL code for single-cycle MIPS Processor

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

USE IEEE.numeric_std.all;

-- VHDL code for the Instruction Memory of the MIPS Processor

entity Instruction_Memory_VHDL is

port (

    pc: in std_logic_vector(15 downto 0);

    instruction: out std_logic_vector(15 downto 0)

    '-3);

end Instruction_Memory_VHDL;


architecture Behavioral of Instruction_Memory_VHDL is

    signal rom_addr: std_logic_vector(3 downto 0);

    -- lw $3, 0($0) -- pc=0

    -- Loop: sltiu $1, $3, 11= pc = 2

    -- beq $1, $0, Skip = 4 --PCnext=PC_current+2+BranchAddr

    -- add $4, $4, $3 = 6

    -- addi $3, $3, 1 = 8

    -- beq $0, $0, Loop--a

    -- Skip c = 12 = 4 + 2 + br

    type ROM_type is array (0 to 15 ) of std_logic_vector(15 downto 0);

    constant rom_data: ROM_type:=

        "1000000110000000",

```

```

"0010110010001011",
"1100010000000011",
"0001000111000000",
"1110110110000001",
"1100000001111011",
"0000000000000000",
"0000000000000000",
"0000000000000000",
"0000000000000000",
"0000000000000000",
"0000000000000000",
"0000000000000000",
"0000000000000000",
"0000000000000000",
"0000000000000000",
"0000000000000000"

```

```

);

```

```

begin

```

```

rom_addr <= pc(4 downto 1);

```

```

instruction <= rom_data(to_integer(unsigned(rom_addr))) when pc < x"0020" else x"0000";

```

```

end Behavioral;

```

VHDL code for the MIPS Processor:

```

library IEEE;

```

```

use IEEE.STD_LOGIC_1164.ALL;

```

```
use IEEE.std_logic_signed.all;
```

```
-- VHDL project: VHDL code for single-cycle MIPS Processor
```

```
entity MIPS_VHDL is
```

```
port (
```

```
    clk,reset: in std_logic;
```

```
    pc_out, alu_result: out std_logic_vector(15 downto 0)
```

```
);
```

```
end MIPS_VHDL;
```

```
architecture Behavioral of MIPS_VHDL is
```

```
    signal pc_current: std_logic_vector(15 downto 0);
```

```
    signal pc_next,pc2: std_logic_vector(15 downto 0);
```

```
    signal instr: std_logic_vector(15 downto 0);
```

```
    signal reg_dst,mem_to_reg,alu_op: std_logic_vector(1 downto 0);
```

```
    signal jump,branch,mem_read,mem_write,alu_src,reg_write: std_logic;
```

```
    signal reg_write_dest: std_logic_vector(2 downto 0);
```

```
    signal reg_write_data: std_logic_vector(15 downto 0);
```

```
    signal reg_read_addr_1: std_logic_vector(2 downto 0);
```

```
    signal reg_read_data_1: std_logic_vector(15 downto 0);
```

```
    signal reg_read_addr_2: std_logic_vector(2 downto 0);
```

```
    signal reg_read_data_2: std_logic_vector(15 downto 0);
```

```
    signal sign_ext_im,read_data2,zero_ext_im,imm_ext: std_logic_vector(15 downto 0);
```

```
    signal JRControl: std_logic;
```

```
    signal ALU_Control: std_logic_vector(2 downto 0);
```

```

signal ALU_out: std_logic_vector(15 downto 0);

signal zero_flag: std_logic;

signal im_shift_1, PC_j, PC_beq, PC_4beq,PC_4beqj,PC_jr: std_logic_vector(15 downto 0);

signal beq_control: std_logic;

signal jump_shift_1: std_logic_vector(14 downto 0);

signal mem_read_data: std_logic_vector(15 downto 0);

signal no_sign_ext: std_logic_vector(15 downto 0);

signal sign_or_zero: std_logic;

signal tmp1: std_logic_vector(8 downto 0);

begin

-- PC of the MIPS Processor in VHDL

process(clk,reset)

begin

if(reset='1') then

    pc_current <= x"0000";

elsif(rising_edge(clk)) then

    pc_current <= pc_next;

end if;

end process;

-- PC + 2

pc2 <= pc_current + x"0002";

-- instruction memory of the MIPS Processor in VHDL

Instruction_Memory: entity work.Instruction_Memory_VHDL

    port map

    (

```

```

        pc=> pc_current,
        instruction => instr
    );
-- jump shift left 1
jump_shift_1 <= instr(13 downto 0) & '0';
-- control unit of the MIPS Processor in VHDL
control: entity work.control_unit_VHDL
    port map
    (reset => reset,
    opcode => instr(15 downto 13),
    reg_dst => reg_dst,
    mem_to_reg => mem_to_reg,
    alu_op => alu_op,
    jump => jump,
    branch => branch,
    mem_read => mem_read,
    mem_write => mem_write,
    alu_src => alu_src,
    reg_write => reg_write,
    sign_or_zero => sign_or_zero
    );
-- multiplexer regdest
reg_write_dest <= "111" when reg_dst= "10" else
    instr(6 downto 4) when reg_dst= "01" else
    instr(9 downto 7);

```

```

-- register file instantiation of the MIPS Processor in VHDL

reg_read_addr_1 <= instr(12 downto 10);

reg_read_addr_2 <= instr(9 downto 7);

register_file: entity work.register_file_VHDL

port map

(

clk => clk,

rst => reset,

reg_write_en => reg_write,

reg_write_dest => reg_write_dest,

reg_write_data => reg_write_data,

reg_read_addr_1 => reg_read_addr_1,

reg_read_data_1 => reg_read_data_1,

reg_read_addr_2 => reg_read_addr_2,

reg_read_data_2 => reg_read_data_2

);

-- sign extend

tmp1 <= (others => instr(6));

sign_ext_im <= tmp1 & instr(6 downto 0);

zero_ext_im <= "0000000000"& instr(6 downto 0);

imm_ext <= sign_ext_im when sign_or_zero='1' else zero_ext_im;

-- JR control unit of the MIPS Processor in VHDL

JRControl <= '1' when ((alu_op="00") and (instr(3 downto 0)="1000")) else '0';

-- ALU control unit of the MIPS Processor in VHDL

ALUControl: entity work.ALU_Control_VHDL port map

```

```

(
    ALUOp => alu_op,
    ALU_Funct => instr(2 downto 0),
    ALU_Control => ALU_Control
);

-- multiplexer alu_src
read_data2 <= imm_ext when alu_src='1' else reg_read_data_2;

-- ALU unit of the MIPS Processor in VHDL
alu: entity work.ALU_VHDL port map
(
    a => reg_read_data_1,
    b => read_data2,
    alu_control => ALU_Control,
    alu_result => ALU_out,
    zero => zero_flag
);

-- immediate shift 1
im_shift_1 <= imm_ext(14 downto 0) & '0';
no_sign_ext <= (not im_shift_1) + x"0001";

-- PC beq add
PC_beq <= (pc2 - no_sign_ext) when im_shift_1(15) = '1' else (pc2 + im_shift_1);

-- beq control
beq_control <= branch and zero_flag;

-- PC_beq
PC_4beq <= PC_beq when beq_control='1' else pc2;

```

```

-- PC_j
PC_j <= pc2(15) & jump_shift_1;

-- PC_4beqj
PC_4beqj <= PC_j when jump = '1' else PC_4beq;

-- PC_jr
PC_jr <= reg_read_data_1;

-- PC_next
pc_next <= PC_jr when (JRControl='1') else PC_4beqj;

-- data memory of the MIPS Processor in VHDL
data_memory: entity work.Data_Memory_VHDL port map
(
    clk => clk,
    mem_access_addr => ALU_out,
    mem_write_data => reg_read_data_2,
    mem_write_en => mem_write,
    mem_read => mem_read,
    mem_read_data => mem_read_data
);

-- write back of the MIPS Processor in VHDL
reg_write_data <= pc2 when (mem_to_reg = "10") else
    mem_read_data when (mem_to_reg = "01") else ALU_out;

-- output
pc_out <= pc_current;
alu_result <= ALU_out;

```

end Behavioral;

VHDL testbench for the MIPS processor:

LIBRARY ieee;

USE ieee.std_logic_1164.ALL;

-- VHDL project: VHDL code for single-cycle MIPS Processor

-- VHDL testbench code for single-cycle MIPS Processor

ENTITY tb_MIPS_VHDL IS

END tb_MIPS_VHDL;

ARCHITECTURE behavior OF tb_MIPS_VHDL IS

-- Component Declaration for the single-cycle MIPS Processor in VHDL

COMPONENT MIPS_VHDL

PORT(

clk : IN std_logic;

reset : IN std_logic;

pc_out : OUT std_logic_vector(15 downto 0);

alu_result : OUT std_logic_vector(15 downto 0)

);

END COMPONENT;

--Inputs

signal clk : std_logic := '0';

signal reset : std_logic := '0';

--Outputs

signal pc_out : std_logic_vector(15 downto 0);

signal alu_result : std_logic_vector(15 downto 0);

```

-- Clock period definitions

constant clk_period : time := 10 ns;

BEGIN

-- Instantiate the for the single-cycle MIPS Processor in VHDL

uut: MIPS_VHDL PORT MAP (

    clk => clk,

    reset => reset,

    pc_out => pc_out,

    alu_result => alu_result

);


-- Clock process definitions

clk_process :process

begin

clk <= '0';

wait for clk_period/2;

clk <= '1';

wait for clk_period/2;

end process;

-- Stimulus process

stim_proc: process

begin

    reset <= '1';

    wait for clk_period*10;

reset <= '0';

```

-- insert stimulus here

wait;

end process;

END;

6. OUTPUT

